AI 101 — build it, ship it, log it — then your instrument

Lecture 1 • Fri 11 Sep 2026 • 14:20–16:20 • R311 IAMS

Two promises for today:
a page that teaches one idea, on the public web, **before the break** •
your own phone app measuring your ESP32 **by the end**

"ship" = publish it where others can open it • "log" = write down what you did, in plain text

Words you will hear today

- **git** — the tool that records a folder's history, change by change
- **repository** — a project folder whose complete history git keeps; on GitHub it also lives online
- **GitHub** — the website where repositories are stored and shared
- **commit** — save a snapshot of your changes with a one-line message
- **push** — upload your commits to GitHub
- **pull request (PR)** — a request to merge your changes into the shared project; someone reviews it first
- **clone** — download a full copy of a repository to your laptop
- **Markdown** — plain text with light formatting (`#` headings, `-` lists)
- **spec** — a specification: what the thing must do, written down before it is built
- **agent** — Claude Code working on your files: reads them, writes code, runs commands, reports back
- **deploy** — publish the page so it is live on the web
- **firmware** — the program that runs on the microcontroller
- **flash** — write the firmware onto the board over USB
- **toolchain** — the compiler and helper programs that turn source code into firmware
- **PlatformIO (`pio`)** — the tool that builds and flashes our firmware
- **KiCad** — the free program we draw circuit boards in
- **access point** — the board's own WiFi network, which your phone joins
- **the check** — one result you can work out by hand before any code exists, which the finished page must reproduce

What we build together

One class instrument — an ESP32-S3 dev board (a small WiFi computer on a board with a USB-C connector and pins) on a class board with **five blocks**, one per student:

- **B1** precision inputs (8 × ±10 V, 16-bit)
- **B2** power supplies
- **B3** signal generation (2 × ±10 V out)
- **B4** isolated switching (4 relays, 2 inputs)
- **B5** digital I/O (5 V logic, trigger)

One repository `class-board-2026`, **one phone app**, **one Python client**.

You read your block, route it (draw its copper tracks), write its driver and demo it.

The board is ordered **Mon 28 Sep** from everyone's pull requests. Boards back ≈ 12 Oct. Demos week of 26 Oct.

Why we start away from the project

AI-assisted work is a **general skill**. The same three moves serve a simulation, a website, a video, a report, a KiCad board, a firmware driver:

Build — from a written spec
Ship — where others can see it
Log — what you did, in plain text

Today, twice:

1. **teach one idea you know** to a high-school student — a page, a simulation, a game. The answer is known before the code exists.
2. the **ESP32 in your hand** — a phone app that measures something. The answer is not known yet.

Electronics design is *one* application of AI, not the syllabus.

Tutor on

Open Claude Code in your clone of `class-board-2026` and type:

```
/tutor L1
```

It asks **which language** (English or Mandarin) and **what you have built before** — type `expert` any time for the whole recipe of a step at once — then checks your toolchain:

`git --version` • `gh auth status` • `pio --version` • the board on a USB port • Cloudflare signed in

It writes your answers into `docs/students/<name>/PROGRESS.md` and creates your branch `e2-<name>` — your course log starts now.

Done when every screen says "toolchain OK". Something missing? Say so now; the tutor fixes it while I talk.

The one-sentence thesis

> An AI agent is a **junior engineer with no memory**.
> It is exactly as good as the project documentation, the task specification and the verification you give it.

Everything today follows from that sentence.

The five working practices

1. **Spec first.** Requirements live in `SPEC.md`, not in the chat window. Ask for a plan before code.
2. **Teach the repository, not the session.** `CLAUDE.md` — the file the agent reads at the start of every session: conventions, pin assignments, build and flash commands, how to test. Written once, loaded every time.
3. **Small, fresh sessions.** One task = one session. Finish, commit, start clean. Long sessions rot.
4. **Handover notes.** `PROGRESS.md` at the end of every session: done / verified / next / gotchas.
5. **AI writes, you verify.** No verification plan → not yours to ship.

Markdown is your lab notebook

`PROGRESS.md`, `SPEC.md`, `notes.md`, `README.md`, this deck, the workbook — all plain text with light formatting.

- git shows **exactly what changed**, line by line, and keeps it next to the code
- it is **searchable** and readable on a phone, on GitHub, in ten years
- the AI **reads it first** at the start of the next session
- the common language of AI work: specs, logs, slides, reports

```
## 2026-09-11 — session 1
- Done: rc-filter teaching page from SPEC.md; deployed to https://rc-filter-xxxx.pages.dev
- Verified: f_c 1591 Hz (pred. 1591.5), −3.0 dB / −45° there; sliders work on my phone
- Next: the teaching questions check the answer; RLC as idea two
- Gotchas: Cloudflare output directory must be "/"
```

"Verify" means a number or an observation

Today	Later
Is f_c where **1/(2πRC)** said it would be?	Does DRC (KiCad's design rule check) say **0**?
Does the noise of an N-sample average fall as **1/√N** on your phone — and if not, why?	Does the scope show the **±5.00 V** sine your block was asked for?

"It looks right" is not verification. "It works" is not verification.
Write the number down. That is the whole trick.

The loop in miniature — today's two exercises

E1a • Build
your topic, your repository • five-line `SPEC.md` — learner outcome + one result worked out by hand before the code exists • plan • one `index.html` • the check • commit

E1b • Ship + log
push • Cloudflare Pages • URL on your phone • `PROGRESS.md` • push → redeploys itself

E2 • Your phone app measures your ESP32
one command out • a mean and its noise back • flash • noise vs N against $1/\sqrt{N}$ — explain the difference • pull request (PR) with the numbers

Nobody leaves without it.

Cloudflare Pages — a free service that turns a repository into a public web page.

E1a — build: teach one idea to a high-school student

Pick a topic you know — EM • electronics • classical mechanics • PID • your research — and build what teaches it.

```
gh repo create <you>/<topic> --public --clone
cd <topic>
code .
```

`gh` = GitHub's command-line tool • `--public` or `--private`, your choice • on the projector the topic is `rc-filter`.

You write `SPEC.md` — five lines:

1. what the learner can do afterwards
2. what the page shows
3. **the check: one result worked out by hand before any code exists** (RC: 1 kΩ, 100 nF → f_c = 1591.5 Hz)
4. a teaching check — a question with a hidden answer
5. one HTML file, plain JavaScript, works on a phone

E1a — plan, run, check, commit

Then tell the agent: *"Read SPEC.md. Plan in five bullets. Then write index.html."*

- **Reject a plan that draws a picture** instead of computing the physics.
- Open the page. **Read your number off it.**
- **Prediction vs result into** `SPEC.md` — with the reason for any disagreement.
- Commit.

Topics with a check: RC filter (f_c) · PID (settling time from the poles) · two charges (field on the axis) · cart-pole (period $2\pi\sqrt{(L/g)} \times \sqrt{(M/(M+m))}$) · orbit ($T^2 \propto a^3$) · standing waves ($f_n$). **Option:** the CSV → scope lab tool. Reference repositories exist for cart-pole and CSV → scope.

Ahead of the room? A real teaching check · a second idea · an explainer video (the lab's show-your-work skills; your own YouTube channel, optional).

E1b — ship + log

1. `git push -u origin main`
2. Cloudflare → **Workers & Pages → Create → Pages → Connect to Git** → your repository → build command *blank*, output `/` → Deploy
3. Open `https://<topic>-xxxx.pages.dev` **on your phone**. Move a slider. Send it to a non-physicist.
4. **You** write `PROGRESS.md`: done / verified / next / gotchas. Commit, push — and watch it redeploy itself.

Fallback: GitHub → Settings → Pages → branch main. That is continuous deployment (every push publishes itself); you just did it. This repository is your course page from now on.

Break

When we come back: join your board's WiFi.

The instrument on your phone

Your board is an **access point** (its own WiFi network). LED blue = the network is up → phone joins `instrument-XXXX` (password `instrument`) → **http://192.168.4.1** → tap *Red*.

firmware on the ESP32-S3
↑↓ one **WebSocket** (a live two-way connection)
phone app (a web page the board serves)
↑↓ same messages
`instrument.py` in Python on a PC

phone → board, one JSON message (plain-text structured data):
`{"block":"base","cmd":"led","args":{"r":255,"g":0,"b":0}}`
board → phone, one reply
board → everyone, **20 × per second**:
status numbers → the chart, the alarms

The two files a control touches

`firmware/src/blocks/base/BaseBlock.cpp`

```
if (strcmp(c, "led") == 0) {
  r_ = a["r"] | r_;  g_ = a["g"] | g_;  b_ = a["b"] | b_;
  showLed();
  reply["r"] = r_; ... return true;
}
// status(): out["counter"] = counter_;
```

`host/pwa/panels/base.js`

```
btn.onclick = () =>
  api.send('base', 'led', { r, g, b });
// onStatus(st):
els.counter.textContent = st.counter;
```

`a["r"] | r_` = the value sent, or the old one if none was sent.

A command is **one `if` and one button**. A number is **one `out[...]` and one `<span>`** (a labelled spot on the page).

E2 — build the measurement, flash it

One command out, one live measurement back — with its noise: the voltage on pin GPIO 4, read by the chip's ADC (analog-to-digital converter). Recipe: workbook § A.4.

1. `BaseBlock.h` : a 4096-sample ring buffer (a fixed-size list that overwrites its oldest entry), `adcN_ = 16`
2. `loop()` : every 1000 µs `analogReadMilliVolts(4)` into the ring — **no** `delay()` , no maths here
3. `handle()` : the command `set_avg {n}` , 1…1024
4. `status()` , 20 × per second: `adc_v` = mean of the last N • `adc_sd` = **noise of one N-sample average** (sd of the N-sample means across the ring) • `avg_n`
5. `panels/base.js` : an **N** input + *Set* → `api.send('base','set_avg',{n})` ; two readouts filled in `onStatus`
6. `pio run -d firmware -e esp32s3-sim -t upload` , then `… -t uploadfs`

`esp32s3-sim` is the build for the bare dev board. `upload` builds the firmware and flashes it; `uploadfs` sends the app's files to the board. The tutor writes the scaffolding; **you** write the statistics, the command and the widget (one control or readout on the phone app).

E2 — verify, then ship it as a pull request

7. **Verify against a prediction:** `adc_sd` for N = 1, 4, 16, 64, 256. Independent samples say `adc_sd(N) = adc_sd(1)/√N` — a factor 16 from 1 to 256. It will not obey exactly. **Say what you see and why.**
8. On your branch `e2-<name>` (your own line of work inside the shared repository, made by the tutor at the start): commit • `git push -u origin e2-<name>` • `gh pr create --fill` • **the table + one sentence + a screenshot in the PR**

Done when your phone sets N and shows the mean and its noise live, and the table — N, measured, predicted — with its explanation is in the PR.

Flash fails? Hold BOOT, tap RST, release, retry • check the USB driver • pick the right port • use a USB-C **data** cable in the connector marked *USB*. Fallback for a late flash failure only: the LED `press` button + counter (the tutor writes it); finish the measurement version in Homework 1.

The class board

Block	What	Parts
B1	8 × ±10 V 16-bit inputs, ADS8688	≈ 32
B2	USB-C / 5 V jack → +5V, +3V3, ±12 V, +5VA	≈ 20
B3	2 × ±10 V outputs, DAC8563 + OPA2192	≈ 15
B4	4 relays, 2 isolated inputs	≈ 24
B5	8 × 5 V logic out, trigger in/out	≈ 15

Base (instructor): dev-board socket, buses, LED, sensor plug, expansion header.

JLCPCB, the factory, assembles it; you solder nothing.

Front panel, **3 × 4 SMA** (small coaxial connectors) at 20 mm:

```
AO1 AO2 TRIG AUX
AI1 AI2 AI3 AI4
AI5 AI6 AI7 AI8
```

AO = analog out • AI = analog in • TRIG = trigger

Your block — volunteers first, then lots

Five names, five blocks. From now on:

- you **read** it (two questions and one design number — Homework 1)
- you **route** it (Lecture 2, 18 Sep: your zone of the PCB, the printed circuit board layout)
- you **write its driver and panel** (Lecture 3, 2 Oct, in sim mode — the firmware fakes its hardware, so it runs on the bare dev board)
- you **measure one number** on it and **demo** it (Oct)

The tutor writes your block into `PROGRESS.md` and opens your block page.

Beyond the baseline

The **hardware is the shared baseline** — fixed by budget and timeline; JLCPCB builds it. The **software, firmware and app are open.** That is where you show what you can do.

Ahead? Ask: *what problem in my lab could this instrument solve?* A data logger to CSV • remote access and an overnight Python sweep • a calibration routine • a PID controller block (today's PID page, made real) • a second board with a camera on a gauge.

Most encouraged: talk to your instrument from LINE or Telegram — a push notification when a value drifts, bot commands that read a value or switch a relay.

Bring a real problem to Lecture 2; the tutor helps you write its `SPEC.md`. It can be the optional fifth item of your demo.

Homework 1

- [ ] **E2 finish + merge** — merge (bring your branch into the main line) once reviewed
- [ ] **E3 read your block** — video V3, two answers **and the design number** in `notes.md`
- [ ] **E4 KiCad ready** — video V4, install KiCad 10 (the part library is in the repository), screenshot your zone
- [ ] **E5 your SPEC paragraph** — what, **the number you will measure**, how you show it
- [ ] **reading** — workbook ch. C (git, KiCad); ch. A where needed

`/tutor HW1` paces it; stuck? ask it for the fix. Email the instructor after that.

Every work session ends with `PROGRESS.md` + commit — your day-1 repository and the class repository alike.

Next Friday: your part of the board

Electronics 101 on **our** schematic (the circuit drawing) · KiCad's six operations · place your missing parts · route your zone · your first PR into the class board

Bring: KiCad 10 installed, the class project open.